

C910 O3 CPU IEW 对齐 — 代码变更总结

总览

- **修改文件:** 15 个
 - **代码变更:** +415 / -47 行
 - **核心改动:** 8 个 C910 风格专用 Issue Queue + Dispatch 路由 + DynInst IQ Entry 字段 + CPU 参数 + RISC-V 架构参数
-

一、C910 IEW 对齐 — Issue Queue 架构改造（5 个核心文件）

1. `src/cpu/o3/inst_queue.hh` — IQ 类型定义 + 接口扩展

新增:

- `enum class IQType { AIQ0, AIQ1, BIQ, LSIQ, SDIQ, VIQ0, VIQ1, VMB }` — 8 个 C910 风格专用 IQ
- `to_string(IQType)` 辅助函数
- `IQUnit::_iqType` 成员 + `iqType()` getter
- `InstructionQueue::findIQByType(type, tid)` — 按类型查找有空位的 IQ
- `InstructionQueue::isFullByType(type, tid)` — 按类型判断是否满
- `InstructionQueue::insertToIQType(inst, type)` — 插入到指定类型 IQ
- `aiqRRCounter + getAIQRRCounter() / advanceAIQRRCounter()` — AIQ0/AIQ1 round-robin 负载均衡

2. `src/cpu/o3/inst_queue.cc` — IQ 实现

新增:

- `parseIQType(string)` — Python 参数字符串 → C++ enum 转换
- `IQUnit` 构造函数中初始化 `_iqType`
- `InstructionQueue` 构造函数中初始化 `aiqRRCounter = 0`
- `findIQByType()` — 遍历 iqs 匹配 type 且有空位
- `isFullByType()` — 同上逻辑取反
- `insertToIQType()` — 完整插入流程（统计 + 依赖追踪 + ready 检查）

修改:

- `insert() / insertNonSpec() / insertToIQType()` — 插入后调用 `inst->setAgeInIQ()` 设置 agevec 位置
- `addIfReady()` — ready 条件从 `readyToIssue()` 改为 `readyToIssue() && !isFrozen()`
 - 对应 C910: `rdy = vld && src0_vld && src1_vld && !frz`

3. `src/cpu/o3/iew.cc` — Dispatch 路由逻辑重写

修改 `dispatchInsts()`，引入 `IQType targetIQ` 变量:

指令类型	目标 IQ	说明
Load	LSIQ	Pipe3 地址计算
Store	LSIQ	地址计算，数据由 LSQ 处理
FENCE	insertBarrier()	保持原始路径（内存序语义）
Nop	不入 IQ	直接标记 issued+executed
Branch/Control	BIQ	含 CALL/RET 标记
Vector memory	VMB	向量访存
Vector mul/div	VIQ1	VFMAU 单元
Vector ALU	VIQ0	向量 ALU
Scalar FP	VIQ1	浮点运算
Int MUL/MADD	AIQ1	MLA 单元
Int DIV	AIQ1	除法
Int ALU	AIQ0/AIQ1	round-robin 负载均衡

CALL/RET 标记：

- `isCall()` → `markProcedureCall()`（标记 pcall）
- `isReturn()` → `markReturnInst()`（标记 rts）

Fallback 路由（目标 IQ 满时）：

目标 IQ	Fallback IQ
AIQ0	AIQ1
AIQ1	AIQ0
BIQ	AIQ0
VIQ0	VIQ1
VIQ1	AIQ0
VMB	LSIQ

Fallback 也失败 → 走通用 `instQueue.insert(inst)` → 再失败则 stall。

4. `src/cpu/o3/dyn_inst.hh` — DynInst 新增 C910 IQ Entry 字段

新增 6 个字段 + 14 个方法：

字段	类型	含义	C910 对应
<code>_frozen</code>	bool	FENCE 同步等待	frz
<code>_procedureCall</code>	bool	函数调用标记	pcall

字段	类型	含义	C910 对应
<code>_returnInst</code>	bool	子程序返回	rts
<code>_iqType</code>	int	分配到的 IQ 类型	—
<code>_ageInIQ</code>	unsigned	agevec 位置	agevec
—	—	src0/src1 就绪	src0_vld / src1_vld

方法列表：

```
isFrozen() / freeze() / unfreeze()
isProcedureCall() / markProcedureCall()
isReturnInst() / markReturnInst()
src0Ready() / src1Ready()
iqType() / setIqType()
ageInIQ() / setAgeInIQ()
```

5. `src/cpu/o3/IQUnit.py` — IQ 类型 Python 定义

新增：

- `class IQType` — 8 种 IQ 类型的 Python 常量
- `iqType` 参数（默认 `"AIQ0"`）

二、C910 IEW 对齐 — CPU 参数调整（1 个文件）

6. `src/cpu/o3/BaseO3CPU.py`

带宽参数修改：

参数	原始值	C910 值
fetchWidth	8	3
decodeWidth	8	3
renameWidth	8	4
dispatchWidth	8	4
issueWidth	8	8（不变）
wbWidth	8	3
commitWidth	8	3

资源参数修改：

参数	原始值	C910 值
----	-----	--------

参数	原始值	C910 值
LQEntries	32	16
SQEntries	32	12
numPhysIntRegs	256	95
numPhysFloatRegs	256	32
numPhysVecRegs	256	64
numROBEntries	192	64

新增：

- 默认 8 个 IQ 实例（AIQ0×11, AIQ1×11, BIQ×12, LSIQ×16, SDIQ×8, VIQ0×10, VIQ1×10, VMB×8）
- 8 个 per-IQ 参数：`aiq0Entries`, `aiq1Entries`, `biqEntries`, `lsiqEntries`, `sdiqEntries`, `viq0Entries`, `viq1Entries`, `vmbEntries`

三、RISC-V 架构参数 — 对齐 C910 实现（7 个文件）

7. `src/arch/riscv/RiscvISA.py` — ISA 配置

参数	原始值	C910 值	说明
<code>enable_rvv</code>	True	False	C910 向量非标准
<code>vlen</code>	256	64	VEC_WIDTH=63 → 64
<code>elen</code>	64	64	FPR_WIDTH=63 → 64
<code>enable_Zicbom_fs</code>	True	False	C910 不支持
<code>enable_Zicboz_fs</code>	True	False	C910 不支持

8. `src/arch/riscv/PMP.py` — PMP 条目数

- `pmp_entries`: 16 → **8**（C910 PMP_REGION_8，固定 8 项）

9. `src/arch/riscv/PMAChecker.py` — PMA 注释

- 添加注释：C910 PMA 通过 BIU/CIU 隐式实现，硬件支持 misaligned

10. `src/arch/riscv/RiscvInterrupts.py` — 中断配置

- `local_interrupt_pins`: 添加 C910 CLINT + PLIC 注释
- `local_interrupt_ids`: 添加 C910 PLIC_INT_NUM=144 注释
- `nmi_cause`: 添加 C910 未实现注释

11. `src/arch/riscv/RiscvTLB.py` — TLB 大小

- `size`: 64 → **1024**（C910 JTLB_ENTRY_1024，可选 2048）

12. `src/arch/riscv/RiscvDecoder.py` — 注释

- 添加 C910 解码由 `ct_ifu_ipdecode.v` 等 IFU 单元实现

13. `src/arch/riscv/RiscvFsWorkload.py` — 注释

- 添加 `semihosting` 等参数为 `gem5` 仿真特性，RTL 无需实现

14. `src/dev/riscv/Clint.py` — CLINT

- `num_threads`: 默认 **1**，添加 C910 `MAX_HART_NUM=32` 注释

15. `src/dev/riscv/Plic.py` — PLIC

- `n_src`: 默认 **144** (C910 `PLIC_INT_NUM=144`)
- `hart_config`: 添加 C910 `PLIC_HART_NUM / PLIC_ID_NUM=10 / PLIC_PRIO_BIT=5` 注释

四、C910 IQ Entry 字段与 C910 对应关系

C910 IQ Entry Structure	gem5 DynInst 字段
<code>vld</code> (entry valid)	<code>isInIQ()</code>
<code>frz</code> (frozen, wait sync)	<code>_frozen</code>
<code>src0_vld</code> (source 0 ready)	<code>src0Ready()</code>
<code>src1_vld</code> (source 1 ready)	<code>src1Ready()</code>
<code>agevec</code> (age arbitration)	<code>_ageInIQ</code>
<code>pcall</code> (procedure call)	<code>_procedureCall</code>
<code>rts</code> (return from subroutine)	<code>_returnInst</code>
<code>iq_type</code> (queue type)	<code>_iqType</code>

五、未修改的部分 (Phase 2-7 后续工作)

Phase	内容	当前状态
Phase 2	多 PRF (Integer/Float/Vector 独立物理寄存器堆)	× 未开始
Phase 3	8 条执行管道 (Pipe0-Pipe7) + FPU/VFMAU/MLA 绑定	× 未开始
Phase 4	C910 专用 stall 类型 (IQ full / LSQ full / FU busy / cache blocked)	× 未开始
Phase 5	Flush 机制 (pipeline flush on mispredict/exception)	× 未开始
Phase 6	前递路径 (forwarding paths) 细化	× 未开始
Phase 7	性能计数器 (C910 PMC)	× 未开始

六、验证结果

- `hello` 二进制在 SE 模式下正常输出 "Hello gem5"

- `baremetal` 二进制在 FS 模式下正常运行（无 UART 输出，符合预期）
- 编译无错误（仅 capstone 库缺失警告，不影响功能）